CoHabitant

Intelligent Shared Living & Expense Management System

DAMG 6210 — Database Management & Database Design | Group 6 | Northeastern University, Spring 2026

Team: **Deep Prajapati · Tianyi Fan · Ashfaq Ahmed Mohd**

🚀 Live on Azure

COHABITANT-CBDBF8F2CHH3A0D5.CENTRALUS-01.AZUREWEBSITES.NET

The Problem — Shared Living is Broken

No single system integrates finance, inventory, chores, and governance for shared living. The result? Conflict, confusion, and cost.

Financial Opacity

Roommates can't track who owes what → unresolved debts pile up silently

Inventory Chaos

No separation between personal and shared items → unauthorized use and disputes

Chore Disputes

Verbal agreements fail — no accountability or proof of task completion

Lease Risk

Manual guest/subletter tracking → compliance violations and cost errors

No Democratic Process

Major household decisions become arguments with no paper trail or resolution

The Gap: No single system integrates finance, inventory, chores, and governance for shared living.

Our Solution — CoHabitant

One Database to Run the Household. CoHabitant is a centralized household management database + Streamlit application built for complete transparency and zero disputes.

Core Capabilities

Splitwise-Style Finance

Equal, custom, and consumption-based splits with pairwise settle-up

AI Receipt Scanner

Gemini 2.5 Flash parses receipts and auto-fills amount, category, split policy

Democratic Governance

Proposal + voting system with digital paper trail for decisions over $50

Chore Accountability

Photo proof required before marking tasks complete

Technology Stack

Frontend

Streamlit · Python 3.11

Database

SQL Server 2022

AI Layer

Google Gemini 2.5 Flash

Infrastructure

Docker · Azure · GitHub Actions

ERD — Conceptual Design (EER)

18 Tables Across 6 Business Domains — designed with EER inheritance to eliminate null attributes and enforce data integrity at the schema level.

EER Inheritance Design

PERSON → LANDLORD + TENANT

Disjoint subtypes avoid null attributes and keep PII clean across user roles

INVENTORY_ITEM → SHARED + PERSONAL

Enforces privacy boundaries and auto-replenishment logic per item type

Key Cardinalities

- **1 LEASE_AGREEMENT → Many TENANTS** — co-signers in shared living arrangements
- **1 EXPENSE → Many EXPENSE_SHARES** — one payer, multiple debtors per transaction
- **1 PROPOSAL → Many VOTES** — one tenant = one vote, full accountability enforced

6 Core Business Domains

👤 User Management

PERSON, LANDLORD, TENANT

🏢 Property & Occupancy

PROPERTY, LEASE_AGREEMENT, SUB_LEASE, GUEST

📦 Inventory

INVENTORY_ITEM, SHARED_ITEM, PERSONAL_ITEM

💰 Finance

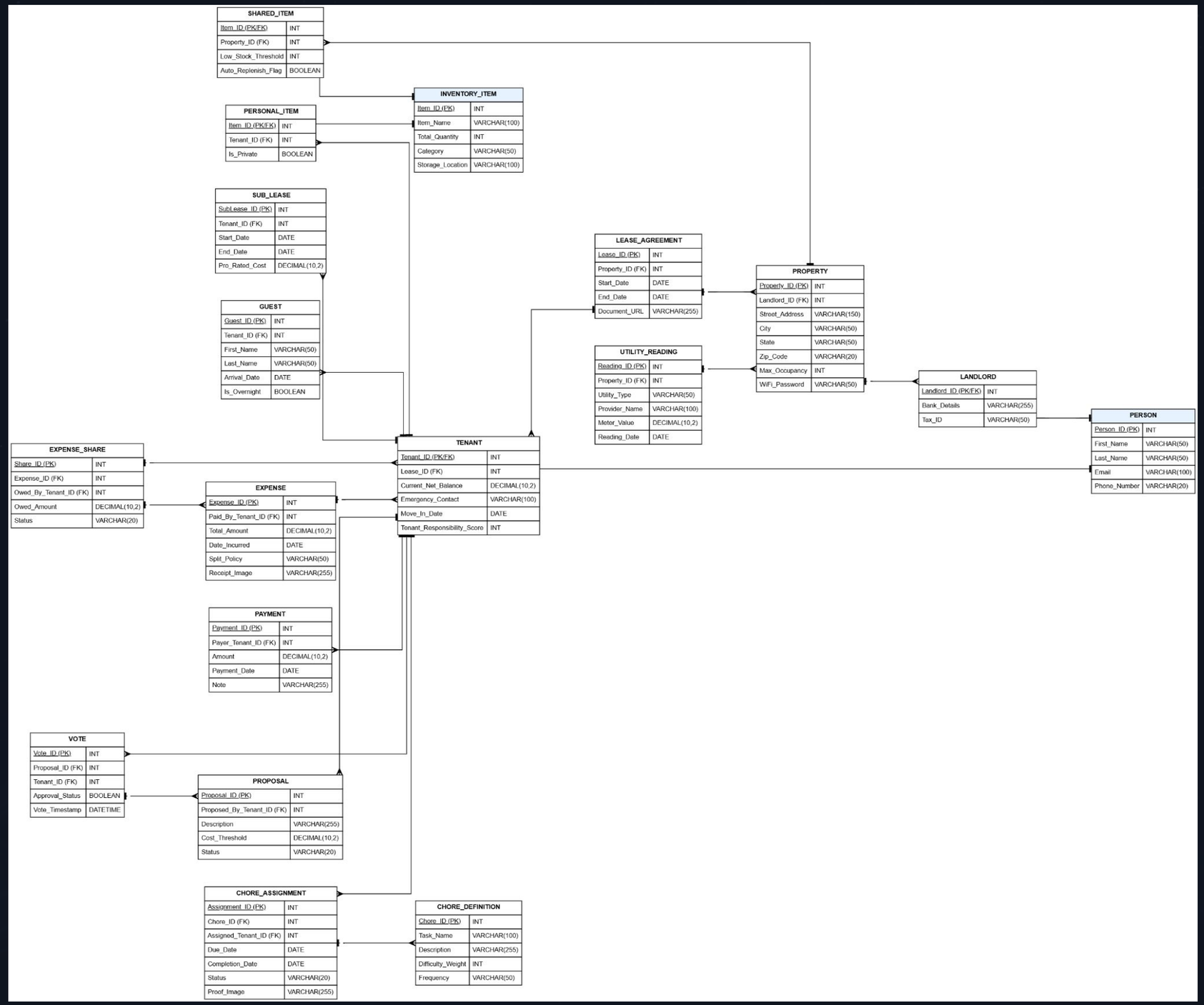
EXPENSE, EXPENSE_SHARE, PAYMENT

🧹 Chores & Governance

CHORE_DEFINITION, CHORE_ASSIGNMENT, PROPOSAL, VOTE

⚡ Utility Analytics

UTILITY_TYPE, UTILITY_READING


SHARED_ITEM
Item_ID (PK/FK) INT
Property_ID (FK) INT
Low_Stock_Threshold INT
Auto_Replenish_Flag BOOLEAN
INVENTORY_ITEM
Item_ID (PK) INT
Item_Name VARCHAR(100)
Total_Quantity INT
Category VARCHAR(50)
Storage_Location VARCHAR(100)
PERSONAL_ITEM
Item_ID (PK/FK) INT
Tenant_ID (FK) INT
Is_Private BOOLEAN
SUB_LEASE
SubLease_ID (PK) INT
Tenant_ID (FK) INT
Start_Date DATE
End_Date DATE
Pro_Rated_Cost DECIMAL(10,2)
GUEST
Guest_ID (PK) INT
Tenant_ID (FK) INT
First_Name VARCHAR(50)
Last_Name VARCHAR(50)
Arrival_Date DATE
Is_Overnight BOOLEAN
LEASE_AGREEMENT
Lease_ID (PK) INT
Property_ID (FK) INT
Start_Date DATE
End_Date DATE
Document_URL VARCHAR(255)
PROPERTY
Property_ID (PK) INT
Landlord_ID (FK) INT
Street_Address VARCHAR(150)
City VARCHAR(50)
State VARCHAR(50)
Zip_Code VARCHAR(20)
Max_Occupancy INT
WiFi_Password VARCHAR(50)
UTILITY_READING
Reading_ID (PK) INT
Property_ID (FK) INT
Utility_Type VARCHAR(50)
Provider_Name VARCHAR(100)
Meter_Value DECIMAL(10,2)
Reading_Date DATE
LANDLORD
Landlord_ID (PK/FK) INT
Bank_Details VARCHAR(255)
Tax_ID VARCHAR(50)
PERSON
Person_ID (PK) INT
First_Name VARCHAR(50)
Last_Name VARCHAR(50)
Email VARCHAR(100)
Phone_Number VARCHAR(20)
EXPENSE_SHARE
Share_ID (PK) INT
Expense_ID (FK) INT
Owed_By_Tenant_ID (FK) INT
Owed_Amount DECIMAL(10,2)
Status VARCHAR(20)
TENANT
Tenant_ID (PK/FK) INT
Lease_ID (FK) INT
Current_Net_Balance DECIMAL(10,2)
Emergency_Contact VARCHAR(100)
Move_In_Date DATE
Tenant_Responsibility_Score INT
EXPENSE
Expense_ID (PK) INT
Paid_By_Tenant_ID (FK) INT
Total_Amount DECIMAL(10,2)
Date_Incurred DATE
Split_Policy VARCHAR(50)
Receipt_Image VARCHAR(255)
PAYMENT
Payment_ID (PK) INT
Payer_Tenant_ID (FK) INT
Amount DECIMAL(10,2)
Payment_Date DATE
Note VARCHAR(255)
VOTE
Vote_ID (PK) INT
Proposal_ID (FK) INT
Tenant_ID (FK) INT
Approval_Status BOOLEAN
Vote_Timestamp DATETIME
PROPOSAL
Proposal_ID (PK) INT
Proposed_By_Tenant_ID (FK) INT
Description VARCHAR(255)
Cost_Threshold DECIMAL(10,2)
Status VARCHAR(20)
CHORE_ASSIGNMENT
Assignment_ID (PK) INT
Chore_ID (FK) INT
Assigned_Tenant_ID (FK) INT
Due_Date DATE
Completion_Date DATE
Status VARCHAR(20)
Proof_Image VARCHAR(255)
CHORE_DEFINITION
Chore_ID (PK) INT
Task_Name VARCHAR(100)
Description VARCHAR(255)
Difficulty_Weight INT
Frequency VARCHAR(50)

Normalization & Key Design Decisions

The schema achieves **Third Normal Form (3NF)** throughout, with deliberate design decisions that prevent data anomalies and support long-term auditability.

Concern	Design Decision
Composite attributes	Name → First_Name + Last_Name; Address → Street, City, State, Zip
EER Inheritance	Table-per-Type: subtypes use supertype PK as PK + FK — no null columns
Financial roles	EXPENSE (Paid_By) ≠ EXPENSE_SHARE (Owed_By) — prevents data loss on role confusion
Vote accountability	VOTE stores Tenant_ID — anonymous voting is architecturally impossible
Utility direction	PROPERTY (1) → UTILITY_READING (M) — corrected from Phase 2 design

⏱️ Temporal Tables

EXPENSE, EXPENSE_SHARE, and PAYMENT use **SQL Server system-versioned temporal tables** for automatic point-in-time audit history — every financial change is permanently recorded.

🗑️ Soft Delete Strategy

An `Is_Active` flag exists on all 18 tables. No data is ever physically removed from the system — full historical record is always preserved for compliance and dispute resolution.

Implementation Highlights — Schema & DDL

Physical Schema: SQL Server 2022 — production-hardened with encryption, indexing, and full error logging.

```
-- Temporal table for full financial audit trail
CREATE TABLE dbo.EXPENSE (
    Expense_ID INT IDENTITY PRIMARY KEY,
    Paid_By_Tenant_ID INT NOT NULL REFERENCES dbo.TENANT(Tenant_ID),
    Total_Amount DECIMAL(10,2) NOT NULL,
    Split_Policy VARCHAR(50) DEFAULT 'Equal',
    Is_Active BIT NOT NULL DEFAULT 1,
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START,
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END,
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime)
) WITH (SYSTEM_VERSIONING = ON
        (HISTORY_TABLE = dbo.EXPENSE_History));
```

5 Filtered Indexes

Optimized on active records using `WHERE Is_Active = 1` for maximum query performance

AES-256 Encryption

LANDLORD PII (Bank_Details, Tax_ID) encrypted via SQL Server certificate chain

Error Logging

SYSTEM_ERROR_LOG table captures all stored procedure failures with full error context

Advanced Features — PSM Layer

The Procedural SQL layer encapsulates all business logic, ensuring the application layer never directly manipulates financial data.

5 Stored Procedures

1

sp_LogExpense

Creates expense + auto-generates shares based on split policy (equal, custom, consumption)

2

sp_SettleExpenseShares

Handles partial/full settlements with deadlock-safe UPDLOCK ordering by ascending Tenant_ID

3

sp_SoftDeleteExpense

Blocks deletion if any share is 'Paid'; only reverses 'Pending' shares to prevent double-refund

Views, UDFs & Trigger

3 Views (Streamlit & Power BI)

`vw_App_Ledger_ActiveBalances` — real-time net balance + pending debts per tenant
`vw_App_Chore_Leaderboard` — completion rates + gamification score
`vw_App_Utility_TimeSeries` — meter readings joined with utility type and property

2 UDFs

`fn_CalculateExpenseShare` — property-scoped equal split math
`fn_GetPendingVoteCount` — returns votes still needed for quorum

1 Trigger

`trg_AfterExpenseInsert` — auto-creates EXPENSE_SHARE rows on new expense insertion, keeping ledger always in sync

Architectural Hardening — Production-Ready

Pre-Deployment Audit: 4 Critical Issues Found & Fixed. Every vulnerability was identified, documented, and resolved before go-live.

Severity	Finding	Fix Applied
🔴 Critical	Partial settlements left Owed_Amount unchanged → over-settlement possible	SP now reduces Owed_Amount by the consumed amount on every partial payment
🟠 High	Soft-delete refunded already-settled 'Paid' shares (double-reverse bug)	Guard blocks deletion if any share is 'Paid'; refund filters on 'Pending' only
🟡 Medium	UPDLOCK acquired in caller/counterparty order → mutual deadlock risk	Deterministic lock ordering by ascending Tenant_ID eliminates deadlock possibility
🔵 Low	Voting page bypassed AppState auth middleware entirely	Migrated to AppState() + auth_gate() for consistent role enforcement

🔐 Security Architecture

Role-Based Access Control via `auth_gate()` middleware enforces Tenant vs. Landlord page-level access throughout the entire application — no page is reachable without authentication and role verification.

Walled Garden Architecture

Property-scoped tenant isolation is enforced at **both the application layer and the database layer**. A tenant in Property A cannot read, write, or infer any data belonging to Property B — by design, not by convention.

Data Insights & Visualizations

Insights derived from seeded data across **3 properties and 9 tenants**. All dashboards built in Power BI and exported as `visualization_report.pdf`.

Financial Dashboard

- Tenant net balance distribution — who is a net creditor vs. debtor
- Pending vs. settled expense share breakdown by category
- Lifetime payment volume per tenant

Utility Time-Series

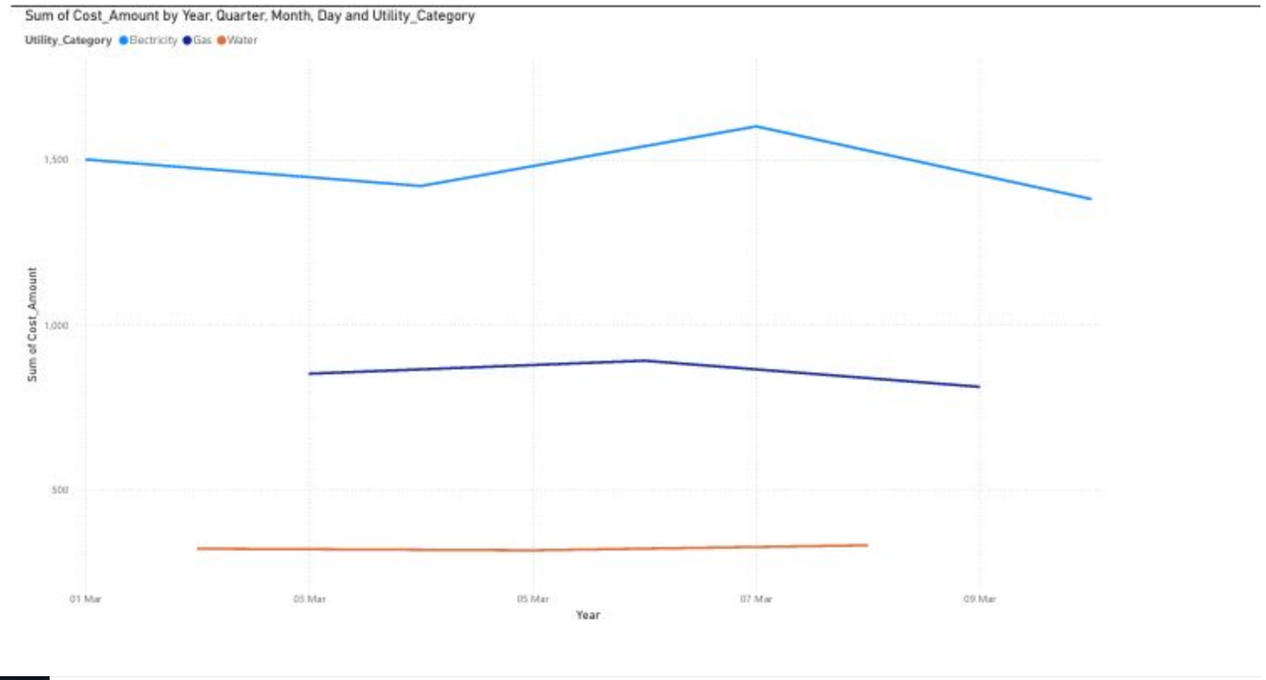
- Monthly meter value trends by utility type (electricity, gas, water, internet)
- Cost per tenant per property over time
- Spike detection for unusual consumption patterns

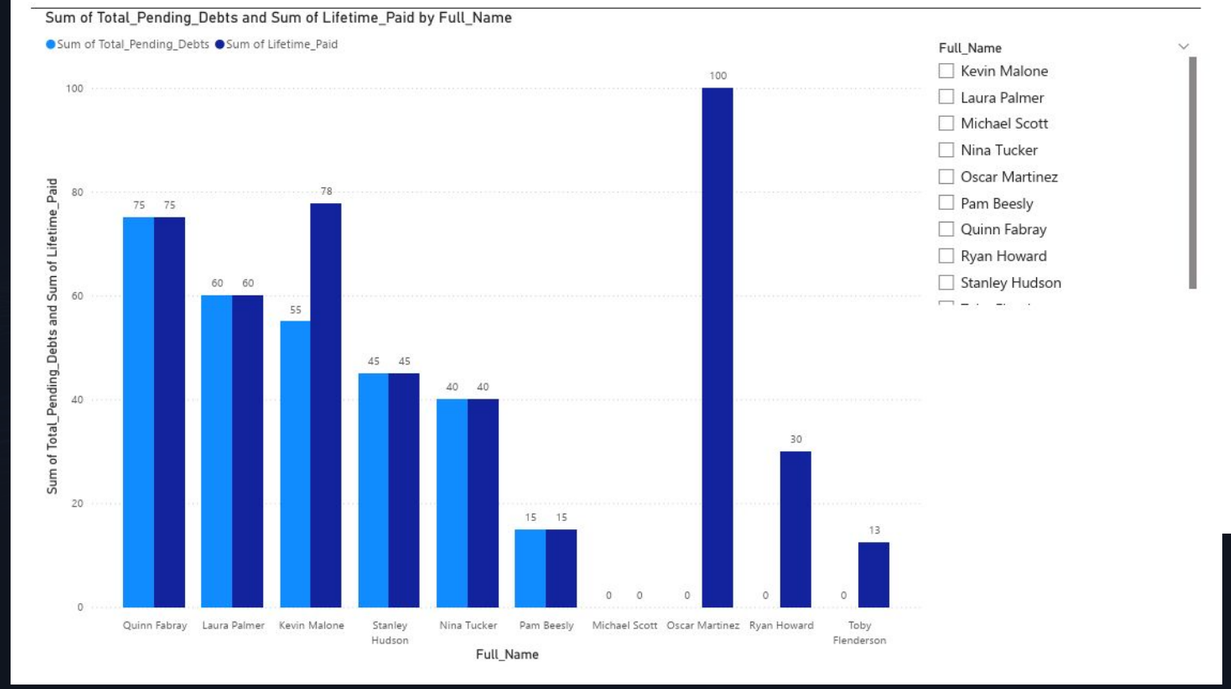
Chore Leaderboard

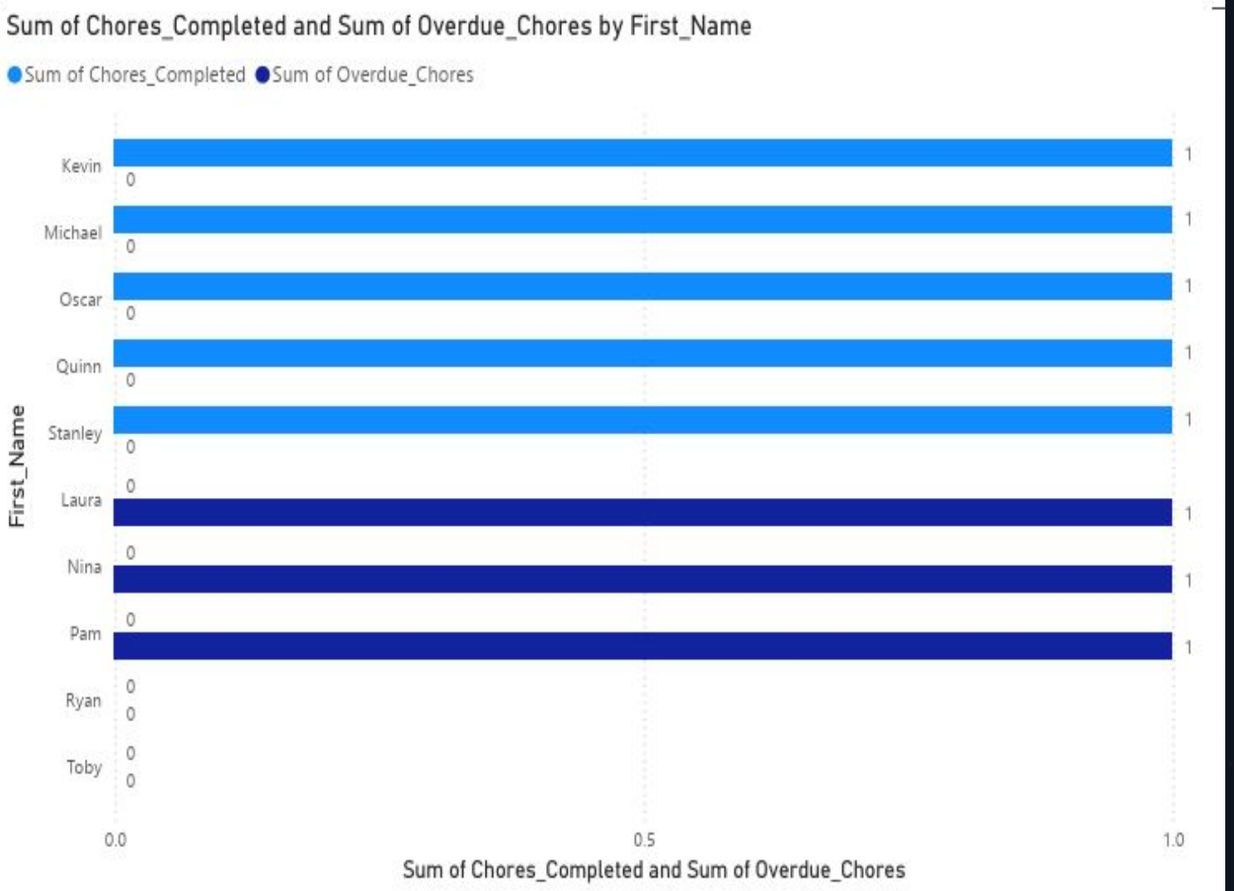
- Completion rate by tenant with difficulty-weighted responsibility score
- Overdue chore count — accountability visualization
- Chore frequency distribution across household tasks

Governance Analytics

- Proposal approval rates — how democratic is the household?
- Voting participation per tenant
- Average proposal cost threshold distribution







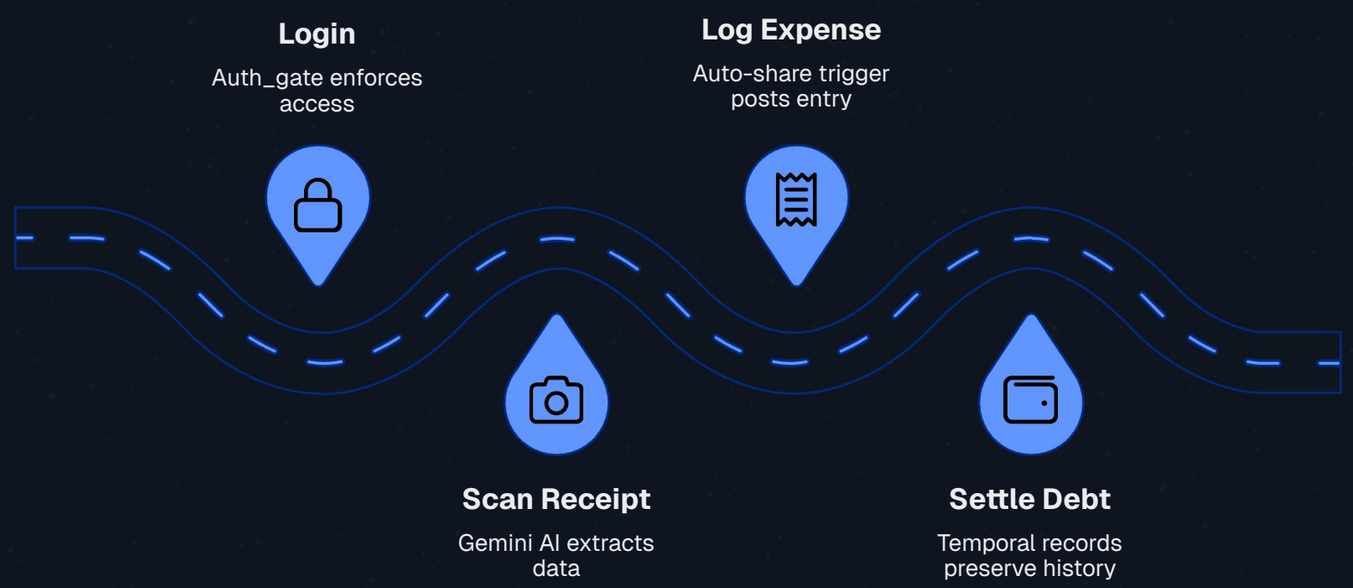
First_Name	Sum of Tenant_Responsibility_Score
Kevin	95
Laura	88
Michael	100
Nina	70
Oscar	92
Pam	85
Total	**892**

Live Demo — The System in Action

Deployed 100% on Azure Free Tier. Every component is live, containerized, and continuously deployed.

Component	Service	Status
Database	Azure SQL Database (Gen5 Serverless, auto-pause 1hr)	✅ Live
App Hosting	Azure App Service F1 Linux	✅ Live
Container	Docker Hub — deep25lelouch/cohabitant	✅ Pushed
CI/CD	GitHub Actions (pytest + Docker build)	✅ Green

🌐 Live URL: cohabitant-cbdbf8f2chh3a0d5.centralus-01.azurewebsites.net



The Value Proposition

One system. Full transparency. Zero disputes.

CoHabitant transforms shared living from a source of conflict into a seamlessly managed, data-driven household — with every financial transaction, chore, vote, and utility reading permanently recorded and instantly accessible.

What Makes It Production-Ready

- Temporal audit trail on all financial tables — dispute-proof by design
- AES-256 encryption on all landlord PII — compliance-ready
- Deadlock-safe stored procedures — concurrent users handled gracefully
- Soft delete across all 18 tables — no data ever lost
- CI/CD pipeline with automated testing — every push is verified